

Setting up GHCR

Getting the GitHub Container Registry publishing your images and letting the VPS pull them. Nine steps, in order — several of them fail if you run them early.

THE THING THAT CATCHES EVERYONE

You cannot configure a package that does not exist yet. The GitHub UI has no "create a package" button — a package is created by the first successful push. So the order is: tag a release → let Actions push → *now* the package exists → set its visibility → then log the VPS in.

Trying to grant the VPS access before step 4 means hunting for a settings page that is not there yet. That is the failure this runbook is ordered to avoid.

GITHUB · REPOSITORY

01

Confirm the workflow is on the default branch

Actions only offers a workflow that exists on the repository's default branch.

`.github/workflows/release.yml` must be committed and pushed to `main` before any tag will trigger it.

```
git add .github/workflows/ci.yml .github/workflows/release.yml
git commit -m "ci: add build/test workflow"
git push origin main
```

Check Actions → the CI workflow runs on that push. If it does not appear at all, the file is not on the default branch or the YAML failed to parse — GitHub reports a parse error on the Actions tab, not on the commit.

02

Allow Actions to write packages

Settings → Actions → General → Workflow permissions. Either is fine:

- *Read and write permissions* — simplest, and the workflow still narrows itself.
- *Read repository contents and packages permissions* — works because `release.yml` declares `permissions: packages: write` for itself.

No PAT is needed for publishing. The workflow authenticates with the built-in `GITHUB_TOKEN`, which exists only for the length of the run — nothing long-lived is stored in the repository or in Actions secrets for this.

03

Cut the first release tag

Tag a commit that CI has already gone green on:

```
git tag v1.0.0
git push origin v1.0.0
```

Watch **Actions** → **Release images**. It builds `spmis-api` and `spmis-web` in parallel and pushes each with three tags: the git tag, a `sha-<short>`, and `latest`.

Owner names are lowercased. GHCR rejects uppercase in an image path. The workflow already does this (`${GITHUB_REPOSITORY_OWNER,,}`), so if your org is `JigawaState` the image is `ghcr.io/jigawastate/spmis-api`. Use the lowercase form everywhere — including `GHCR_OWNER` in the VPS `.env`.

04

Confirm the packages now exist

Two packages appear: `spmis-api` and `spmis-web`. They are **private by default**. Open each and confirm the tag you just pushed is listed.

A personal account's packages live at `github.com/users/<you>/packages`; an organisation's at `github.com/orgs/<org>/packages`. If you cannot find them, the push step failed — read the Actions log rather than looking harder.

05

Link each package to the repository

Package → **Package settings** → *Manage Actions access* → add the repository with **Write**. This lets later workflow runs push new versions of a package that already exists.

Usually already set, because the package was created *by* a workflow in that repository. It is worth confirming: the symptom of it being wrong is a **denied** on the second release, not the first.

06

Decide visibility — private is the right default

CHOICE

WHAT IT MEANS

Private

RECOMMENDED

The VPS authenticates with a read-only token (step 7). Your image layers stay unreadable to strangers.

Public

No login needed on the VPS. Anyone can pull and inspect the image.

A public image is a public filesystem. It holds no secrets — the stack reads config from a root-owned `.env` on the VPS, never from the image — but it does expose your exact dependency versions, which is a free vulnerability inventory for anyone deciding whether to bother with you. Keep them private; the extra step is one `docker login`, once.

07

Create a read-only token for the VPS MOST COMMON FAILURE

Developer settings → Personal access tokens → Tokens (classic) → Generate new token (classic).

- Scope: `read:packages` only. Nothing else. Not `write:packages`, not `repo`.
- Expiry: set one, and put the renewal date in a calendar.
- Copy it now — GitHub will not show it again.

An expiring token fails silently in the worst way. Everything keeps running on the images already on the host; the failure only appears the next time you deploy — which is usually the moment you are trying to ship a fix. If your uptime monitor cannot tell you the token expired, the calendar reminder is the control.

A fine-grained token also works if you give it read access to packages, but classic with a single scope is easier to audit: one token, one permission, obviously harmless if leaked.

08

Log the VPS in — once

```
# Pass the token on stdin so it never reaches shell history or `ps`.
echo "ghp_XXXXXXXXXXXXXXXXXXXX" | docker login ghcr.io \
-u <your-github-username> --password-stdin
```

Expect `Login Succeeded`. The credential is written to `~/.docker/config.json`.

That file stores the token base64-encoded, not encrypted. Base64 is an encoding, not a secret. Log in as the user that runs the stack, and lock the file down:

```
chmod 600 ~/.docker/config.json
```

If you run compose under `sudo`, note that root has a *different* `~/.docker/config.json`. Logging in as your user and then pulling as root is the classic "it worked yesterday" failure.

Pull, and prove it works

Set the two values in the VPS `.env`, then pull:

```
# /srv/spmis/.env - root-owned, gitignored
GHCR_OWNER=jigawastate      # lowercase
IMAGE_TAG=v1.0.0           # the tag you pushed in step 3

# Then, from the compose directory:
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

A clean `pull` of both `spmis-api` and `spmis-web` is the proof that every step above is correct. Carry on with `docs/DEPLOY.md` §2.3 onward.

When it fails

WHAT YOU SEE	WHAT IT ACTUALLY IS
<code>denied: denied</code> on pull	Not logged in, or logged in as a different OS user than the one running compose. Re-run step 8 as that user.
<code>manifest unknown</code>	<code>IMAGE_TAG</code> names a tag that was never pushed. Check the package's Versions list against your <code>.env</code> .
<code>invalid reference format</code>	An uppercase letter in <code>GHCR_OWNER</code> .
<code>unauthorized</code> after months of working	The PAT expired. Issue a new one and repeat step 8.
Actions: <code>denied: permission_denied</code>	Step 2 — workflow permissions, or step 5 — the package is not linked to the repository.
No packages exist after a tag push	The workflow did not run. Confirm the tag matches <code>v*</code> and that <code>release.yml</code> is on the default branch.

Rolling back

This is why releases are tagged. Every previous release is still a tag in GHCR, so a rollback is an `.env` edit and a pull — no rebuild, no branch surgery:

```
IMAGE_TAG=v1.0.0          # the previous good release
docker compose -f docker-compose.prod.yml pull
docker compose -f docker-compose.prod.yml up -d
```

Rolling back *code* does not roll back the *database*. If the bad release ran a migration, review it before rolling back — a schema the old code does not understand is a worse outage than the one you are fixing.

Companion to `docs/DEPLOY.md` §2.2 and the CI/CD section. Images are built by `.github/workflows/release.yml` ; the VPS only ever pulls them.